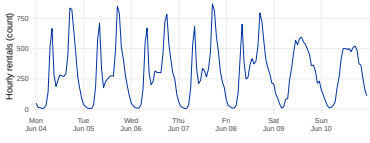
Applied Machine Learning

Time Series
Introduction and Forecasting Setup



Learning goals

- What is a time series?
- Tree-based models for time series
- Detrending and multi-step forecasting

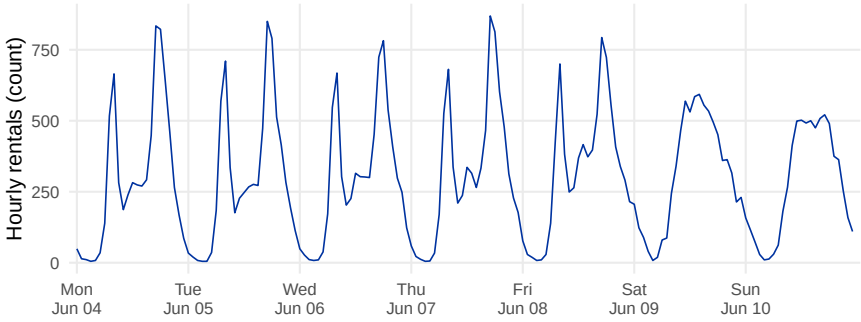
RUNNING EXAMPLE: THE BIKE SHARING DATASET I



Hourly bike rental data from Washington D.C.

holiday	workingday	weather	temp	humidity	windspeed	datetime	count
False	False	clear	9.84	0.81	0.00	2011-01-01 00:00	16
False	False	clear	9.02	0.80	0.00	2011-01-01 01:00	40
False	False	clear	9.02	0.80	0.00	2011-01-01 02:00	32
False	False	clear	9.84	0.75	0.00	2011-01-01 03:00	13
False	False	clear	9.84	0.75	0.00	2011-01-01 04:00	1

WHAT DOES THE DATA LOOK LIKE? I



Hourly bike rentals – one week in June 2012. Notice: **daily seasonality** (commuter peaks at 8am/5pm on weekdays) and **weekly seasonality** (smooth single-peak pattern on Sat/Sun).

KEY DEFINITIONS I



- **Time series** $\{y_t\}_{t=1}^{T}$: ordered observations indexed by time:
  - Timestamps - Specific instants in time.
  - Fixed periods - Such as the whole month of January 2017, or the whole year 2020.
  - Intervals of time - Indicated by a start and end timestamp. Periods can be thought of as special cases of intervals.
  - Experiment or elapsed time - Each timestamp is a measure of time relative to a particular start time (e.g., the diameter of a cookie baking each second since being placed in the oven), starting from 0.
- **Exogenous variable** / **covariate**: external feature $x_t^{(j)}$ influencing y_t
- **Trend**: long-term increase or decrease
- **Seasonality**: systematic, calendar-related patterns
- **Stationarity** (briefly): distribution of y_t does not change over time

TASKS I



- **Time Series Analysis**
  Decompose time series into trend, seasonality, and residuals for understanding → time series course
- **Time Series Forecasting**
  Fit a model on historical data and make predictions about the future.
- **Time Series Classification**
  Classify entire time series recordings, e.g., classify sensor signals as normal vs. faulty.
- **Anomaly Detection**
  Identify unusual observations or subsequences, e.g., fraud detection, sensor fault monitoring.

HOW TO TACKLE TIME SERIES FORECASTING? I



- **Statistical Models**
  ARIMA, ETS (Error, Trend, Seasonality) → time series course
- **Recurrent Models**
  Deep learning architectures that track the state of data over time → deep learning lecture
- **Transformer Models**
  Attention-based architectures for sequential data → deep learning lecture
- **Feature Engineering + Standard ML Models**
  Engineer temporal features (lags, rolling stats, calendar) from the time index and feed them to standard ML models (e.g., gradient boosting). → **focus of this lecture**

WHY TREE-BASED MODELS FOR TIME SERIES? I



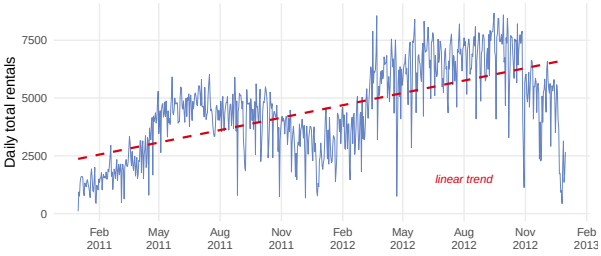
With proper feature engineering, standard tabular ML models work surprisingly well for time series forecasting:

- Handle mixed feature types naturally (calendar integers + continuous weather data)
- No stationarity assumptions required (unlike ARIMA)
- Fast to train and easy to tune
- Feature engineering makes temporal patterns accessible to non-temporal models
- Very competitive in Kaggle competitions and industry (e.g., gradient-boosted trees were widely used in top M5 forecasting competition solutions, often in ensembles[1])

Caveat: Tree-based models cannot extrapolate beyond the training range. If the data has a strong upward trend, predictions will plateau. See the next slide for how to handle this.

[1] https://mofc.unic.ac.cy/m5-competition/

DETRENDING FOR TREE-BASED MODELS I



1. Fit a trend model on the **training fold only** (e.g., linear regression on the time index)
2. Subtract the trend from training targets: $r_t = y_t - \hat{\text{trend}}_t$
3. Train the tree model on the residuals r_t
4. At prediction time: extrapolate the trend, predict residuals, add them: $\hat{y}_t = \hat{r}_t + \hat{\text{trend}}_t$

Important: this is a target transformation and must be part of the modelling pipeline. The trend must be re-estimated on each training fold; detrending on the full dataset before splitting leaks future information and makes CV results overly optimistic.
Alternative: use differencing ($y_t - y_{t-1}$) as the target, which also removes linear trends.

MULTI-STEP FORECASTING STRATEGIES I



How to predict multiple steps ahead (e.g., the next 168 hours)?

- **Recursive:** train a 1-step model, feed its prediction as input for the next step, repeat
  - Simple, but errors accumulate over the horizon
- **Direct:** train a separate model for each horizon ($h = 1, 2, \ldots, H$)
  - No error accumulation, but H models to train and no shared learning
- **Multi-output:** single model predicts all H horizons at once
  - Efficient, but requires a model that supports vector output
- **DirRec (hybrid):** train a direct model for each horizon, but include recursive predictions from earlier horizons as additional features